Campus Ride:

A Campus Auto-Booking and Ride-Pooling Platform

Arpita Girdhar
1024030778

Trijal Mittal
1024030784

Tushit Kumar
1024031079

Department of Computer Science and Engineering
Thapar Institute of Engineering and Technology

Submitted to: Dr. Jeelani Asif
Course: UCS503
August 17, 2026

1 Introduction and Problem Statement

Getting around the Thapar campus by shared auto-rickshaw is something almost every student relies on daily, but the process of actually finding one is entirely informal. Students wait at fixed points hoping an auto with free seats happens to pass by, drivers have no visibility into where demand is building up elsewhere on campus, and there is no way to know in advance whether a seat will even be available. This leads to long, unpredictable waits at some stops while autos elsewhere sit idle or under-filled, and to disputes over fare and change that a fixed, published per-seat rate would remove entirely.

Campus Ride is proposed to close this gap. It is a mobile application, built with Flutter so the same codebase runs on Android and the web, that digitizes campus auto booking for both riders and drivers. Students pick a pickup and drop stop and are automatically matched to an available auto, either one already on the road passing their stop, or the next one waiting in a queue, while drivers get a simple dashboard showing their current trip, passenger list, and queue position. The goal is to replace guesswork and informal waiting with a shared, visible booking system, the same way ride-hailing apps replaced hailing a cab on the street.

2 Objectives

The primary goal of this project is to build a two-sided mobile application that digitizes campus auto booking, automatically pools students into autos already en route where possible, and gives drivers a simple queue-based dispatch system, replacing today's informal wait-and-hope process with one structured and visible platform.

Specific objectives:

- Let students book a seat by selecting a pickup and drop stop from a fixed list of campus locations, with a live seat-availability check before they confirm.

- Automatically pool a new booking onto an auto that is already en route and passes both the student's pickup and drop stop, only pulling a fresh auto from the idle queue when no such match exists.
- Run a fair, cyclic queue for idle drivers, so the next available auto is always the one that has been waiting longest.
- Verify that only genuine Thapar students can book, and that drivers can register and authenticate with just their phone number.
- Show the student their driver's name, phone number, vehicle number, and UPI QR code once a booking is confirmed, so payment is direct and fare disputes are eliminated by a fixed, published per-seat rate.

3 Methodology

Campus Ride is a software engineering project, so the methodology here is the system design: the technology used, how the application is broken into modules, how those modules work together, and how the finished system will be tested.

3.1 System Architecture and Tech Stack

The system is built across three layers:

- Frontend (Flutter): A single Dart codebase producing both an Android app and a web build, with two separate flows: one for students, one for drivers.
- Backend (Node.js / Express): REST APIs that handle location data, driver and student registration, the pooling/matching logic, and booking and payment-status records.
- Database (SQLite via better-sqlite3): Stores campus locations, students, drivers, autos, and bookings.
- Authentication (Firebase): Google Sign-In restricted to @thapar.edu accounts for students, and phone-number OTP verification for drivers.

Because the client is built in Flutter, the application runs on an Android phone and in a browser from the same source, which keeps development effort focused on one codebase instead of two separate native apps.

3.2 Module Development

The application is broken into four core modules:

- Authentication: Students sign in with Google, restricted to the university email domain; drivers verify their identity with a one-time SMS code sent to their phone number. Both sessions persist locally so a returning user lands straight on their dashboard instead of signing in again.
- Matching and Booking: Given a pickup stop, drop stop, and seat count, the server first checks for an auto already en route in the correct direction with enough free seats and a route that covers the requested stops; if none qualifies, the next auto in the idle queue is dispatched instead. Every booking records its fare, seats, and payment status.

- Driver Dashboard: Shows the driver whether they are idle (with their position in the queue) or on a trip (with a live passenger manifest), and lets them mark a trip complete, which returns their auto to the back of the queue.
- Student Booking Flow: Lets a student check live availability as they choose stops, book a seat, view their assigned driver's contact details, vehicle number, and UPI QR code, confirm payment, and later review a full history of past trips.

3.3 Execution and Integration

Development happened feature by feature against a locally running backend, with each piece (authentication, the booking flow, the driver dashboard, and payment display) built and manually tested end to end before moving to the next, so integration issues surfaced immediately rather than at the end. The backend and Flutter client were developed and tested together throughout, using an Android emulator and a browser build in parallel to catch platform-specific issues (such as networking differences between web and Android) early.

3.4 Testing and Validation

The system is checked against specific, observable criteria rather than just “does it run”:

- Matching correctness: a new booking is only pooled onto an en-route auto if that auto's remaining capacity, direction, and route genuinely cover the requested pickup and drop.
- Queue fairness: idle autos are dispatched in the order they became idle, and a completed trip correctly returns an auto to the back of the queue.
- Access control: student registration is rejected for any email outside the @thapar.edu domain, and a driver cannot proceed past registration without a valid phone verification.
- Session persistence: a signed-in student or driver who closes and reopens the app lands directly on their dashboard, without repeating login.

Testing was done manually against these criteria on both the Android emulator and a browser build, using test accounts for students and a Firebase test phone number for drivers during development.

3.5 Constraints and Safety

The interface has to stay simple enough for a quick booking between classes, so both the student and driver flows are deliberately kept to a small number of screens and taps. There are no physical safety concerns since this is a booking and coordination tool rather than something controlling the vehicle itself, but there are data-safety and trust concerns worth planning for: a driver's phone number and UPI QR code are only shown to a student once they have an active booking with that driver, not published openly. To fit the project timeline, a few features are intentionally left out of this version and treated as possible extensions rather than core scope: an in-app payment gateway with server-verified transactions (the current version treats payment as the student self-confirming they paid the driver directly via UPI), push notifications, and driver ratings. None of these affect the core booking and matching workflow; they can be added later without changing the system's architecture.

4 Timeline

Week	Task	Deliverable
1–2	Project setup, Firebase configuration, backend schema design	Flutter project scaffold, working local backend
3–5	Build authentication (Google Sign-In for students, phone OTP for drivers)	Working sign-in for both roles, persisted sessions
6–8	Build the matching/pooling engine and student booking flow	Live availability check, seat booking, driver assignment
9–10	Build the driver dashboard and student payment/history screens	Queue view, passenger manifest, driver QR display, booking history
11	End-to-end testing across web and Android	Bug fixes, verified matching and queue behaviour
12	Buffer week: deployment prep, final polish and documentation	Final submission

5 Resources and Budget

Material resources: No specialized hardware is needed. The project uses Firebase's free Spark tier for authentication during development (with Google Sign-In already functional; real SMS delivery for driver OTPs requires upgrading to Firebase's pay-as-you-go Blaze plan before a live rollout), a local SQLite database for the prototype backend, and free-tier hosting (e.g., Render) planned for deployment.

Budget: No external funding is required for development or a small pilot. Every tool and service used, authentication, database, and hosting, is available on a free tier; the only anticipated future cost is Firebase SMS charges beyond the free monthly quota once phone verification moves from test numbers to real student and driver traffic.